

JENSEN YRKESHÖGSKOLA

DevOps — Examensarbete

March 19, 2026

When Does Kubernetes Become Worth It?

*Measuring Reliability, Stability, and Execution Overhead
in Workflow Orchestration Systems Moving from
Shared VM to Isolated Kubernetes Execution*

Author:

[Your Full Name]

Supervisor:

[Supervisor Name]

JENSEN Yrkeshögskola

Program: DevOps

Academic Year: 2024–2025

Abstract

This thesis investigates how reliability, system stability, and execution performance change when a workflow orchestration system is migrated from a single virtual machine (VM) using a shared process executor to a Google Kubernetes Engine (GKE) cluster using the Kubernetes Run Launcher with isolated pod execution. *Dagster* is used as a representative modern workflow orchestration framework. The findings are intended to be directionally relevant for the broader class of pipeline orchestration systems that execute concurrent workloads.

The central research problem is that engineering teams routinely make VM-to-Kubernetes migration decisions based on industry assumptions rather than measured behavioral evidence. The primary contribution of this thesis is the identification and empirical measurement of a *crossover point* — formally defined as the concurrent workload level at which the VM deployment’s job success rate drops below 95% and the GKE deployment’s total execution time including scheduling overhead becomes lower than the VM’s execution time under active resource contention.

Five research questions are investigated through six controlled experiments measuring job success rate, execution time variance, failure blast radius, autoscaling response time, Kubernetes scheduling overhead, and infrastructure reproducibility across four defined workload levels. Experiments are repeated three times per level and results reported with standard deviation to ensure statistical credibility.

The study is grounded in four literature pillars: resource contention and performance collapse theory, Kubernetes overhead research, autoscaling response behavior, and distributed systems fault isolation. Infrastructure is provisioned entirely through CDKTF in Python and managed via Helm to ensure experimental reproducibility.

Keywords: Kubernetes, GKE, Dagster, workflow orchestration, resource contention, pod isolation, autoscaling, crossover point, CDKTF, reliability measurement, DevOps

Acknowledgements

[Write your acknowledgements here. Thank your supervisor, company, and anyone who supported the work.]

Contents

Abstract	ii
Acknowledgements	iii
List of Tables	vii
List of Figures	viii
Abbreviations	ix
1 Introduction	1
1.1 Background and Motivation	1
1.2 The Research Problem	1
1.2.1 The Surface Problem	1
1.2.2 The Deeper Problem — The Actual Research Gap	2
1.2.3 The Literature Gap Statement	2
1.3 Purpose	2
1.4 The Core Idea	3
1.5 Primary Contribution — The Crossover Point	3
1.6 Research Hypothesis	3
1.7 Research Questions	4
1.8 How the Questions Form One Argument	5
1.9 Practical Value and Target Audience	5
1.10 Delimitations	5
1.11 Thesis Structure	6
2 Literature Review	7
2.1 Pillar 1 — Resource Contention and Performance Collapse	7
2.2 Pillar 2 — Kubernetes Performance Overhead at the Infrastructure Level	7
2.3 Pillar 3 — Autoscaling Behavior and Response Latency	8
2.4 Pillar 4 — Fault Isolation and Blast Radius in Distributed Systems	9
2.5 Positioning Statement — The Gap This Thesis Fills	9
3 Method and Experimental Design	11

3.1	Research Design	11
3.2	Operational Definitions	11
3.3	Workload Definition	12
3.4	Infrastructure	13
3.4.1	VM Deployment (Baseline)	13
3.4.2	GKE Deployment (Comparison)	13
3.4.3	CDKTF Stack	13
3.5	Experimental Design	13
3.5.1	Experiment 1 — VM Failure Threshold (answers SQ ₁)	13
3.5.2	Experiment 2 — GKE Pod Isolation Under Equivalent Load (answers SQ ₂)	14
3.5.3	Experiment 3 — Failure Blast Radius Test (answers SQ ₂)	14
3.5.4	Experiment 4 — Autoscaling Response Under Spike (answers SQ ₃)	14
3.5.5	Experiment 5 — GKE Overhead and Crossover Point (answers SQ ₄)	15
3.5.6	Experiment 6 — Infrastructure Reproducibility (answers SQ ₅)	15
3.6	Metrics Summary	15
3.7	Validity Statement	16
4	Results	18
4.1	Experiment 1 Results — VM Failure Threshold (SQ ₁)	18
4.2	Experiment 2 Results — GKE Pod Isolation (SQ ₂)	18
4.3	Experiment 3 Results — Blast Radius Test (SQ ₂)	18
4.4	Experiment 4 Results — Autoscaling Response (SQ ₃)	19
4.5	Experiment 5 Results — Overhead and Crossover Point (SQ ₄)	19
4.6	Experiment 6 Results — Infrastructure Reproducibility (SQ ₅)	19
5	Discussion	20
5.1	Discussion of SQ ₁ — The VM Failure Threshold	20
5.2	Discussion of SQ ₂ — Pod Isolation Effectiveness	20
5.3	Discussion of SQ ₃ — Autoscaling Response	20
5.4	Discussion of SQ ₄ — The Crossover Point	20
5.5	Discussion of SQ ₅ — Infrastructure Reproducibility	21
5.6	Trade-off Analysis	21
5.7	Limitations	21
6	Conclusions	22
6.1	Summary of Findings	22
6.2	Answer to the Main Research Question	22
6.3	Practical Recommendations	22

6.4 Future Research	22
A CDKTF Infrastructure Code	24
B Helm Values — Dagster Kubernetes Run Launcher	25
C GitHub Actions Pipeline	26
D Experiment Data Templates	27

List of Tables

3.1	Defined Workload Levels	12
3.2	Complete Metrics Table	15
4.1	VM Deployment Results by Workload Level	18
D.1	Experiment 1 Data Collection Template — VM Baseline	27

List of Figures

Abbreviations

Abbreviation	Meaning
CDKTF	Cloud Development Kit for Terraform
CPU	Central Processing Unit
DORA	DevOps Research and Assessment
GCP	Google Cloud Platform
GKE	Google Kubernetes Engine
HPA	Horizontal Pod Autoscaler
IaC	Infrastructure as Code
MTTR	Mean Time To Recovery
SLI	Service Level Indicator
SLO	Service Level Objective
SRE	Site Reliability Engineering
VM	Virtual Machine

1 Introduction

1.1 Background and Motivation

Modern data engineering relies on pipeline orchestration frameworks to automate, schedule, and monitor workflows across variable and unpredictable workloads. A common initial deployment strategy is to host such a framework on a single virtual machine (VM) using a shared process executor. While simple to configure, this architecture contains a fundamental flaw: all pipeline jobs execute within the same process on the same host, sharing a single pool of CPU and memory.

As concurrent job execution increases, resource contention arises. Jobs compete for the same resources, performance degrades non-linearly, and under sufficient load the system fails entirely. Critically, this degradation does not scale gradually — it accelerates as contention approaches capacity limits, producing a collapse effect rather than a smooth performance curve (**queueing_theory**).

Dagster is used in this thesis as a representative modern workflow orchestration system. It provides a Kubernetes Run Launcher that dispatches each pipeline run as an isolated pod on a GKE cluster, where each pod receives defined CPU and memory resources. Failures are theoretically contained within individual pods rather than propagating across the system, and the cluster can expand dynamically through the Horizontal Pod Autoscaler (HPA) and GKE cluster autoscaler when demand increases.

While this architectural migration is well-documented at the tool level, there is limited empirical evaluation of its actual effectiveness under controlled workload conditions.

1.2 The Research Problem

1.2.1 The Surface Problem

A VM-hosted workflow orchestration system becomes unreliable under concurrent workload because all jobs share the same compute resources. This is an observable and known failure mode.

1.2.2 The Deeper Problem — The Actual Research Gap

The surface problem is not the research gap. The research gap exists at the intersection of two bodies of literature that have not yet been connected.

Existing research evaluates Kubernetes performance and autoscaling behavior at the *infrastructure level* — measuring container overhead, pod scheduling latency, and cluster autoscaler response under generic workloads (**k8s__overhead__study**). Separately, research on workflow orchestration systems focuses on scheduling algorithms, DAG optimization, and system architecture (**airflow__study**). What is missing is empirical measurement of how infrastructure-level Kubernetes mechanisms — pod isolation, resource limits, and horizontal autoscaling — actually affect *application-level* workflow execution behavior under controlled concurrent workload conditions.

As **k8s__overhead__study**<empty citation> demonstrate, Kubernetes introduces measurable overhead at the infrastructure level. As resource contention theory predicts, shared execution environments collapse non-linearly under load (**queueing__theory**). But the specific workload threshold at which migrating from VM-based to Kubernetes-based workflow execution changes the balance between those two forces has not been empirically identified.

This thesis bridges that gap by measuring system behavior at the pipeline execution level rather than at the infrastructure level alone.

1.2.3 The Literature Gap Statement

Existing studies focus on Kubernetes performance and autoscaling at the infrastructure level, but there is limited empirical evidence on how these mechanisms affect application-level workflow orchestration systems under controlled concurrent workload conditions. This thesis addresses that specific gap.

1.3 Purpose

The purpose of this thesis is to design, implement, and evaluate a controlled experimental comparison of single-VM and GKE-based *Dagster* pipeline execution, measuring how reliability, system stability, and execution performance change under increasing concurrent workload, and identifying the specific workload threshold at which the GKE migration becomes net beneficial.

1.4 The Core Idea

This thesis measures how reliability, failure behavior, execution performance, and system stability change under increasing concurrent workload when a workflow orchestration system moves from a shared execution model on a single VM to an isolated distributed model on Kubernetes. It identifies the specific workload level at which that migration becomes net beneficial.

This thesis is not a proof that Kubernetes is better than a VM. It is not a Kubernetes performance benchmark, a tool comparison, or a cloud provider comparison. It focuses specifically on behavioral differences between two architectural execution models — shared process execution versus isolated pod execution — measured at the workflow job execution level under controlled conditions.

1.5 Primary Contribution — The Crossover Point

The primary contribution of this thesis is the identification and measurement of a **crossover point**, formally defined as:

The concurrent workload level at which **(1)** the VM deployment’s job success rate drops below 95% due to resource contention, **AND (2)** the GKE deployment’s total execution time including pod scheduling overhead and container startup latency becomes lower than the VM’s execution time under active contention.

This crossover point is not an interpretation — it is a computable result derived directly from experimental data. It gives engineers a specific, data-backed threshold for when the Kubernetes migration becomes net beneficial.

1.6 Research Hypothesis

Migrating workflow orchestration execution from a single-VM process executor to a Kubernetes Run Launcher on GKE will produce measurable reliability improvements through pod isolation and scalability improvements through horizontal autoscaling, but will also introduce measurable scheduling latency and execution overhead that does not exist in the VM deployment.

These advantages are *conditional* and depend on workload characteristics, system configuration, and resource constraints. At low concurrency levels, the VM may outperform GKE on execution speed because resource contention has not yet reached critical levels and there is no pod scheduling overhead to pay. At high concurrency levels, GKE is

expected to outperform the VM as pod isolation prevents the contention collapse that a shared execution model cannot avoid.

If results show the VM performs better across all tested concurrency levels, that is an equally valid and valuable finding.

1.7 Research Questions

Main Research Question

How do reliability, system stability, and execution performance change when a workflow orchestration system is migrated from a single-VM process executor to a Kubernetes Run Launcher on GKE under increasing concurrent workload, and at what specific workload level does this migration become net beneficial as measured by the formally defined crossover point?

Supporting Questions

SQ₁ — VM Failure Threshold and Degradation Profile

At what concurrency level does a single-VM workflow deployment fail, and how do job success rate, execution time variance, and resource utilization degrade as concurrent load increases toward that threshold?

SQ₂ — Pod Isolation Effectiveness and Blast Radius Containment

To what degree does Kubernetes pod isolation contain failures and reduce execution time variance compared to the single-VM deployment under equivalent concurrent workload levels?

SQ₃ — Autoscaling Response Speed and Job Impact During Scale-out

How quickly does the GKE cluster provision additional capacity when submitted workload exceeds current node resources, and how many workflow jobs are delayed or fail during the scale-out window before new capacity becomes available?

SQ₄ — Operational Overhead and the Crossover Point

What measurable scheduling latency and execution overhead does Kubernetes introduce compared to the VM process executor at each concurrency level, and at what concurrency level does this overhead become smaller than the reliability gain from pod isolation — the formally defined crossover point?

SQ₅ — Infrastructure Reproducibility

Can the complete GKE cluster and *Dagster* deployment be rebuilt identically across multiple cycles using CDKTF and Helm, and what deviations if any

arise between cycles that could affect experimental validity?

1.8 How the Questions Form One Argument

The five supporting questions are not independent. They form a single connected logical chain.

SQ₁ establishes the exact failure profile of the VM architecture — the baseline evidence. SQ₂ tests whether GKE actually fixes what SQ₁ measured as broken. SQ₃ measures the most operationally critical behavior of GKE under stress — the autoscaling window during which jobs are unprotected. SQ₄ synthesizes SQ₁ through SQ₃ into the crossover point — the primary contribution. SQ₅ validates the experimental environment, which is a prerequisite for all other findings to be credible.

1.9 Practical Value and Target Audience

These findings give small teams, startups, and engineers a data-based answer to the question: *at what workload scale does migrating a workflow orchestration system from a VM to Kubernetes become worth the added complexity and overhead?*

1.10 Delimitations

This study does not include:

- Paid GCP services beyond the free tier
- Comparison with other cloud platforms such as AWS EKS or Azure AKS
- Comparison between orchestration tools — the focus is on execution model behavior, not tool selection
- ML model accuracy evaluation
- Multi-process VM configurations — only the default process executor is used
- Real or sensitive data — synthetic workloads only

Fallback: If GKE free tier prevents stable cluster operation, a minimal single-node Kubernetes configuration will be used instead. This constraint will be documented and analyzed as a finding in SQ₃ and SQ₄ rather than treated as a study failure.

1.11 Thesis Structure

Chapter 2 reviews the four literature pillars underpinning this study. Chapter 3 defines the method, experimental design, and metrics. Chapter 4 presents the results of all six experiments. Chapter 5 discusses the findings in relation to the research questions and the crossover point. Chapter 6 presents conclusions, practical recommendations, and directions for future research.

2 Literature Review

This chapter reviews four interconnected research areas that together provide the theoretical foundation for the experimental design. Each pillar connects directly to one or more supporting questions.

2.1 Pillar 1 — Resource Contention and Performance Collapse

(Theoretical basis for SQ_1)

Resource contention arises when multiple processes compete for finite shared resources — most commonly CPU cycles and memory — on a single host. Unlike bandwidth-limited degradation, resource contention causes non-linear throughput collapse as load approaches system capacity (**queueing_theory**).

Classic queueing theory (**little_law**) demonstrates that as utilisation approaches 100%, response time approaches infinity non-linearly. For a shared process executor, this means that adding one additional concurrent job beyond the system’s stable operating region does not produce a proportional decrease in throughput — it produces a disproportionately large degradation, which can cascade into complete system failure.

In the context of workflow orchestration, this manifests as individual jobs failing due to CPU starvation or memory exhaustion, which in a shared execution model can cause the orchestration process itself to crash, taking all other running jobs with it. This is the failure mode that SQ_1 empirically measures and quantifies for the specific workload profiles defined in this study.

Research connection: SQ_1 identifies the empirical concurrency threshold at which non-linear collapse occurs for *Dagster* pipeline workloads on a single VM.

2.2 Pillar 2 — Kubernetes Performance Overhead at the Infrastructure Level

(Theoretical basis for SQ_4)

A substantial body of research has evaluated the performance overhead introduced by containerization and Kubernetes orchestration compared to bare-metal or VM execution

(**k8s_overhead_study**; **container_performance**).

The primary sources of overhead in a Kubernetes execution model relevant to this study are pod scheduling latency — the time between a job being submitted and a pod being assigned to a node — and container startup time — the time between pod assignment and the first application instruction executing. These overheads are well-documented at the infrastructure level but have not been measured at the application-level workflow execution layer for pipeline orchestration systems specifically.

Additionally, Kubernetes introduces control plane communication overhead and network namespace isolation costs that do not exist in a native VM process. For short-duration jobs, these fixed costs represent a proportionally larger fraction of total execution time than for long-running jobs.

Research connection: SQ_4 extends infrastructure-level overhead measurement to the application workflow execution layer and identifies at what concurrency level this overhead is outweighed by the reliability gains from pod isolation — the crossover point.

2.3 Pillar 3 — Autoscaling Behavior and Response Latency

(Theoretical basis for SQ_3)

Kubernetes HPA scales pod count based on observed CPU or memory metrics, while GKE cluster autoscaler provisions new nodes when pod scheduling demand exceeds current node capacity (**k8s_autoscaler**). Both mechanisms introduce reaction latency that is absent from a fixed-capacity VM deployment.

Research on autoscaling behavior in cloud environments consistently shows that autoscaling is not instantaneous (**autoscaling_latency**). The reaction window — the period between a load spike exceeding current capacity and new capacity becoming available for scheduling — typically ranges from tens of seconds to several minutes depending on cluster configuration and node pool provisioning time. During this window, newly submitted jobs cannot be scheduled and either queue, delay, or fail depending on system configuration.

For workflow orchestration systems, this window has significant operational consequences: jobs submitted during the autoscaling gap are unprotected, and depending on the orchestration framework’s retry behavior, they may fail permanently or re-queue after significant delay. This hidden cost of autoscaling is well-documented at the infrastructure level but has not been measured specifically for workflow pipeline workloads.

Research connection: SQ_3 measures the autoscaling response window and its job-level

impact specifically for *Dagster* pipeline workloads under a defined spike scenario.

2.4 Pillar 4 — Fault Isolation and Blast Radius in Distributed Systems

(Theoretical basis for SQ_2)

A foundational principle of distributed systems design is that failures should be contained within the smallest possible boundary (**blast_radius**). The concept of blast radius — the scope of a system impacted by a single failure event — is central to reliability architecture in microservices and cloud-native systems (**microservices_reliability**).

In a single-VM shared execution environment, the blast radius of any individual job failure is potentially the entire system. A job that exhausts memory can trigger out-of-memory process termination that affects all other jobs running on the same host. In a Kubernetes pod model, each job runs in an isolated namespace with enforced resource limits, meaning a failing pod is terminated by the OOM killer without directly affecting other pods on the same node.

This theoretical blast radius reduction is one of the primary architectural motivations for the Kubernetes migration. Whether this reduction holds in practice at the workflow execution level — and by how much it reduces execution time variance and improves job success rate for concurrently running pipelines — is an empirical question that existing literature at the infrastructure level does not answer for application-level workflow systems.

Research connection: SQ_2 empirically tests the blast radius reduction at the workflow execution level, measuring whether pod isolation contains failures and reduces execution time variance under equivalent concurrent workload conditions.

2.5 Positioning Statement — The Gap This Thesis Fills

Each of the four pillars above has been studied independently. Pillar 1 establishes contention theory. Pillar 2 measures Kubernetes overhead at infrastructure level. Pillar 3 documents autoscaling response behavior. Pillar 4 provides the theoretical basis for pod isolation benefits.

What is missing is a study that connects these four pillars at the *application-level workflow execution layer* under controlled concurrent workload conditions. This thesis occupies that intersection, measuring how infrastructure-level mechanisms produce

application-level behavioral outcomes for workflow orchestration systems under load, and identifying the specific workload threshold at which the architectural migration produces a net benefit.

3 Method and Experimental Design

3.1 Research Design

This study follows a **controlled experimental design** with a before-and-after comparison structure. The baseline condition is *Dagster* deployed on a single VM using the default process executor. The comparison condition is *Dagster* deployed on GKE using the Kubernetes Run Launcher. Identical workloads are applied to both conditions and results are compared across all defined metrics.

The goal is not to create perfectly controlled identical environments, but to measure the combined behavioral effect of the architectural difference under realistic deployment conditions. This reflects how VM-to-Kubernetes migration decisions are made in practice.

3.2 Operational Definitions

Reliability Three measurable properties: (1) *job success rate* — the percentage of *Dagster* runs completing without error at each concurrency level; (2) *failure isolation* — whether one failing job measurably affects other concurrently running jobs; (3) *MTTR* — time in seconds from failure detection to successful job re-execution.

System Stability

Measured primarily through *execution time variance* — the standard deviation of job execution time across repeated runs at each concurrency level. Variance reveals whether system behavior is consistent under load, which mean performance metrics cannot capture. A system with high variance is operationally unstable even if its mean execution time is acceptable.

Scalability Two measurable properties: (1) *concurrency capacity* — maximum concurrent jobs the system sustains without success rate dropping below 95%; (2) *autoscaling response time* — time in seconds from spike workload submission to new GKE node availability.

Operational Overhead

Three measurable components: (1) *pod scheduling latency* — time from *Dagster* job submission to pod entering running state; (2) *container*

startup time — time from pod running state to first job instruction executing; (3) *net execution time delta* — difference in total end-to-end execution time between VM and GKE for identical jobs at identical concurrency levels.

Crossover Point

Formally defined as the concurrent workload level at which **(1)** the VM job success rate drops below 95% due to resource contention, **AND (2)** the GKE total execution time including overhead becomes lower than the VM execution time under active contention. This is a computable result derived directly from experimental data, not an interpretation.

Infrastructure Reproducibility

The degree to which three consecutive CDKTF destroy-and-rebuild cycles produce identical cluster state, measured by configuration comparison and post-rebuild workload execution consistency.

3.3 Workload Definition

Workloads are defined precisely to ensure experimental reproducibility and to simulate realistic pipeline execution resource consumption patterns.

Table 3.1: Defined Workload Levels

Level	Concurrent Jobs	Per-Job CPU	Per-Job Mem- ory	Job Type
Light	1	0.25 CPU	256 MB	Simple data loop
Medium	3	0.50 CPU	512 MB	Data processing
Heavy	5	1.00 CPU	1 GB	CPU-bound computation
Spike	10	0.50 CPU	512 MB	Burst exceeding node capacity

Both CPU-bound and memory-bound job variants are included because they trigger different failure modes: CPU contention degrades performance gradually while memory exhaustion causes sudden job termination. The spike level is specifically designed to exceed current node capacity and trigger the GKE autoscaler.

Each workload level is run **three times** per environment. Results are reported as mean and standard deviation across runs to account for natural system variability.

3.4 Infrastructure

3.4.1 VM Deployment (Baseline)

- Google Cloud Compute Engine instance (e2-standard or equivalent free-tier)
- *Dagster* deployed with default process executor
- No resource isolation between jobs
- No autoscaling capability

3.4.2 GKE Deployment (Comparison)

- GKE cluster provisioned via CDKTF in Python
- *Dagster* configured with Kubernetes Run Launcher via Helm
- Per-pod resource requests and limits defined per workload level
- HPA enabled for pod-level scaling
- GKE cluster autoscaler enabled for node-level scaling
- All infrastructure defined as code for reproducibility

3.4.3 CDKTF Stack

The entire GKE infrastructure — node pools, networking, IAM roles, and Helm releases — is defined in Python using CDKTF. This ensures the environment can be destroyed and recreated identically, which is a prerequisite for the reproducibility validation in Experiment 6.

GitHub Actions is used for CI/CD-driven deployment. This is not framed as a GitOps implementation but as a standard automated deployment mechanism.

3.5 Experimental Design

3.5.1 Experiment 1 — VM Failure Threshold (answers SQ₁)

Setup: Deploy *Dagster* on the VM with default process executor.

Procedure: Run light, medium, heavy, and spike workload levels. Each level is repeated three times. Record job success rate, mean execution time, execution time variance, CPU utilisation, memory utilisation, and MTTR at each level.

Goal: Identify the exact concurrency point at which success rate drops below 95% and

document the shape of the degradation curve leading to that point.

3.5.2 Experiment 2 — GKE Pod Isolation Under Equivalent Load (answers SQ₂)

Setup: GKE cluster with Kubernetes Run Launcher. Identical resource profiles to Experiment 1.

Procedure: Run identical workload levels as Experiment 1, three repetitions each. Record same metrics.

Goal: Calculate the delta in job success rate, execution time variance, and MTTR between VM and GKE at each concurrency level to quantify what changes when jobs move from shared to isolated execution.

3.5.3 Experiment 3 — Failure Blast Radius Test (answers SQ₂)

Setup: Medium workload running on both VM and GKE in separate test runs.

Procedure: After three concurrent jobs are running, deliberately crash one job pod mid-execution using `kubectl delete pod` on GKE and `kill -9` on the VM. Measure the number of other running jobs affected, time until surviving jobs recover, and their success rate.

Goal: Test whether pod isolation's theoretical blast radius reduction holds in practice under real concurrent workload conditions.

3.5.4 Experiment 4 — Autoscaling Response Under Spike (answers SQ₃)

Setup: GKE cluster at initial capacity insufficient for the spike workload level.

Procedure: Submit 10 concurrent jobs simultaneously, exceeding current node capacity. Measure: (1) time from submission to new node becoming available; (2) number of jobs delayed during scale-out; (3) number of jobs that fail rather than wait; (4) time for full cluster recovery to baseline state. Repeated three times to account for GCP provisioning variability.

Goal: Quantify the autoscaling response window and its impact on workflow jobs during the unprotected period.

3.5.5 Experiment 5 — GKE Overhead and Crossover Point (answers SQ₄)

Setup: Both VM and GKE environments running identical single jobs for overhead isolation.

Procedure: At each concurrency level, record pod scheduling latency and container startup time on GKE. Calculate net execution time difference between VM and GKE per job per level. Plot both overhead and contention cost against concurrency level.

Goal: Identify the crossover point using the formal definition: the level at which the VM success rate drops below 95% AND GKE total execution time becomes lower than VM execution time under contention.

3.5.6 Experiment 6 — Infrastructure Reproducibility (answers SQ₅)

Setup: GKE cluster and full *Dagster* deployment via CDKTF and Helm.

Procedure: Destroy and recreate the complete environment three consecutive times using CDKTF destroy and apply. After each rebuild, run the light workload scenario and record results. Compare cluster configuration state and behavioral output across all three cycles.

Goal: Validate that the experimental environment produces identical results across rebuild cycles. Any deviations are documented as findings rather than treated as errors.

3.6 Metrics Summary

Table 3.2: Complete Metrics Table

Metric	Definition	VM	GKE	Answers
Job success rate	% runs completing without error	✓	✓	SQ1, SQ2
Mean execution time	End-to-end time per run	✓	✓	SQ1, SQ2, SQ4
Execution time variance†	Std deviation under load	✓	✓	SQ1, SQ2
CPU utilisation	Per-job CPU consumption	✓	✓	SQ1, SQ2
Memory utilisation	Per-job memory consumption	✓	✓	SQ1, SQ2

(continued from previous page)

Metric	Definition	VM	GKE	Answers
MTTR	Failure to successful re-run	✓	✓	SQ1, SQ2
Failure blast radius	Does one failure affect others?	✓	✓	SQ2
Autoscale response time	Spike to new node ready		✓	SQ3
Jobs delayed in scale-out	Waiting during autoscale window		✓	SQ3
Jobs failed in scale-out	Failed rather than waited		✓	SQ3
Pod scheduling latency	Submission to pod running		✓	SQ4
Container startup time	Pod running to job executing		✓	SQ4
Net execution time delta	VM vs GKE per job per level	✓	✓	SQ4
Crossover point [‡]	Level where GKE becomes net better	✓	✓	SQ4
Infra reproduction accuracy	Config identical across 3 rebuilds		✓	SQ5

[†] Primary stability metric. [‡] Primary contribution — formally computable from experimental data.

3.7 Validity Statement

The VM and GKE environments are not identical. They differ in hardware, scheduling mechanisms, networking, and resource allocation. The goal is not to eliminate these environmental differences but to measure their combined effect as part of real system behavior, which is how architectural migration decisions are made in practice.

All experiments use *Dagster* as the representative workflow orchestration system and synthetic workloads under GCP free tier constraints. Results may not generalize directly to all workflow systems or all cloud environments. However, the behavioral patterns identified — the failure degradation curve, the blast radius reduction, the autoscaling response window, and the crossover point — are expected to be directionally relevant for similar pipeline orchestration frameworks under comparable concurrency conditions.

If results are inconsistent across experimental runs, that inconsistency is documented

and analyzed as a finding rather than suppressed as error.

4 Results

This chapter will present the quantitative results of all six experiments after the experimental phase is complete. Each section corresponds to one experiment and one or more supporting questions.

4.1 Experiment 1 Results — VM Failure Threshold (SQ₁)

[Insert results tables and figures here after experiments are run.]

Example table structure:

Table 4.1: VM Deployment Results by Workload Level

Level	Success Rate (%)	Mean Time (s)	Variance (s ²)	CPU (%)	Memory (MB)
Light					
Medium					
Heavy					
Spike					

4.2 Experiment 2 Results — GKE Pod Isolation (SQ₂)

[Insert results here.]

4.3 Experiment 3 Results — Blast Radius Test (SQ₂)

[Insert results here.]

4.4 Experiment 4 Results — Autoscaling Response (SQ₃)

[Insert results here.]

4.5 Experiment 5 Results — Overhead and Crossover Point (SQ₄)

[Insert results here. The crossover point will be reported here as the primary empirical finding.]

4.6 Experiment 6 Results — Infrastructure Reproducibility (SQ₅)

[Insert results here.]

5 Discussion

This chapter will discuss all findings in relation to the research questions and existing literature after results are collected. The structure below defines what each section must address.

5.1 Discussion of SQ₁ — The VM Failure Threshold

This section will discuss the measured failure threshold against resource contention theory from Pillar 1. It will address whether the collapse was non-linear as predicted and at what concurrency level it occurred. It will also discuss whether the result was surprising and what it implies for teams currently running similar workloads on a VM.

5.2 Discussion of SQ₂ — Pod Isolation Effectiveness

This section will evaluate whether pod isolation delivered the theoretical blast radius reduction predicted by distributed systems fault isolation theory (Pillar 4). It will compare execution time variance between VM and GKE and discuss whether the isolation held under deliberate failure conditions.

5.3 Discussion of SQ₃ — Autoscaling Response

This section will discuss the measured autoscaling response window against the autoscaling latency literature from Pillar 3. It will evaluate whether the response time makes GKE viable under spike conditions and what job-level impact was observed during the scale-out gap.

5.4 Discussion of SQ₄ — The Crossover Point

This section will present and interpret the primary contribution — the formally defined crossover point. It will discuss at what concurrency level the point was found, what it means for engineering migration decisions, and how it relates to Kubernetes overhead literature from Pillar 2. It will also address whether the result was surprising or consistent with theoretical predictions.

5.5 Discussion of SQ₅ — Infrastructure Reproducibility

This section will evaluate the reproducibility of the CDKTF and Helm setup and discuss any deviations observed across rebuild cycles and their potential impact on experimental validity.

5.6 Trade-off Analysis

This section will synthesise all five research question discussions into a balanced trade-off analysis: what the GKE migration improves, what it costs, and under what conditions each architecture is more appropriate. This section answers the main research question directly.

5.7 Limitations

- Environments are not identical — treated as part of the behavioral comparison, not eliminated
- GCP free tier resource constraints may affect results — documented as a research variable
- Synthetic workloads — may not capture all real-world pipeline behavior patterns
- Single cloud provider — results directionally relevant but may not generalize to all platforms
- *Dagster* as representative system — other workflow frameworks may behave differently

6 Conclusions

6.1 Summary of Findings

[To be completed after experimental phase. This section will summarise each finding in one paragraph per supporting question and then state the primary contribution — the crossover point — explicitly.]

6.2 Answer to the Main Research Question

[To be completed after experimental phase. This section directly answers the main research question: how does system behavior change and at what workload level is migration net beneficial?]

6.3 Practical Recommendations

Based on the experimental findings, the following recommendations are offered for engineering teams making VM-to-Kubernetes migration decisions for workflow orchestration systems:

crossover point finding

1. Consider autoscaling response window when designing spike-handling strategies
2. Use CDKTF or equivalent IAC tooling to ensure environment reproducibility before migration
3. Measure execution time variance, not just mean performance, when evaluating system stability

6.4 Future Research

The most natural extension of this thesis is the investigation of how these findings hold when workloads are real NLP transformer-based jobs with variable and unpredictable resource demand patterns. This forms the proposed direction for future master’s level research.

Specifically, the master’s thesis would deploy Dagster on the GKE platform established here, run real NLP pipeline jobs including text ingestion, embedding generation, and

model inference, and investigate whether the crossover point identified in this thesis shifts under the specific resource variability patterns of transformer-based workloads.

Additional directions include:

- Comparison of crossover points across different workflow frameworks (Airflow, Prefect)
- Investigation of the effect of node pool configuration on autoscaling response time
- Extension to multi-tenant GKE environments

A CDKTF Infrastructure Code

Paste your main CDKTF Python stack code here.

Listing A.1: CDKTF Stack — GKE Cluster Definition

```
1 from constructs import Construct
2 from cdktf import App, TerraformStack
3 from cdktf_cdktf_provider_google.provider import GoogleProvider
4 from cdktf_cdktf_provider_google.container_cluster import
   ContainerCluster
5
6 class GKEStack(TerraformStack):
7     def __init__(self, scope: Construct, id: str):
8         super().__init__(scope, id)
9
10        GoogleProvider(self, "google",
11                        project="your-project-id",
12                        region="europe-north1"
13        )
14
15        ContainerCluster(self, "dagster-cluster",
16                          name="dagster-gke",
17                          location="europe-north1-a",
18                          initial_node_count=1,
19                          # ... additional configuration
20        )
21
22 app = App()
23 GKEStack(app, "dagster-gke-stack")
24 app.synth()
```

B Helm Values — Dagster Kubernetes Run Launcher

Paste your Helm values file here.

Listing B.1: Dagster Helm values.yaml — Kubernetes Run Launcher

```
1 Example structure      replace with your actual values
2 dagster:
3   runLauncher:
4     type: K8sRunLauncher
5     config:
6       k8sRunLauncher:
7         jobNamespace: dagster
8         envConfigMaps:
9           - dagster-instance-envs
10        resources:
11          requests:
12            cpu: "500m"
13            memory: "512Mi"
14          limits:
15            cpu: "1000m"
16            memory: "1Gi"
```

C GitHub Actions Pipeline

Paste your GitHub Actions workflow YAML here.

D Experiment Data Templates

Table D.1: Experiment 1 Data Collection Template — VM Baseline

Run	Level	Success %	Mean (s)	StdDev (s)	CPU %	Mem (MB)
1	Light					
2	Light					
3	Light					
1	Medium					
2	Medium					
3	Medium					
1	Heavy					
2	Heavy					
3	Heavy					
1	Spike					
2	Spike					
3	Spike					